

MODULE 1 ASSIGNMENT

“The Journey of a Web Page Request Across the Internet”

Student Name: Jose Bijoy

Roll No: 41

Class: CSA

Course: PCCST501 – Computer Networks

Module: 1

Website chosen: google.com (Google Search)

1. Introduction

When someone types `www.google.com` into their browser and hits Enter, a fast chain of operations kicks off across multiple layers of the Internet's protocol architecture. Even though the whole process usually finishes in well under a second, it involves resolving a name into an address, setting up a connection, exchanging data securely, and finally rendering the page — with each of these jobs handled by a different layer of the TCP/IP stack working in coordination. This report walks through that sequence in detail, using a Google Search request as a running example.

2. The TCP/IP Protocol Stack

The TCP/IP model splits network communication into four layers. Each layer performs a specific function and passes data down to the layer beneath it (when sending) or up to the layer above (when receiving).

Layer	Protocols	Function
Application Layer	HTTP/HTTPS, DNS, FTP, SMTP	Formats and requests the actual web page data
Transport Layer	TCP, UDP	Ensures reliable, ordered, error-checked delivery
Internet Layer	IP, ICMP	Adds source/destination IP addresses; routes packets
Network Access Layer	Ethernet, Wi-Fi (802.11), ARP	Converts packets to physical signals (frames/bits)

Figure 1: The four layers of the TCP/IP protocol stack

What each layer does

Application Layer – This is where the browser lives. It builds the HTTP/HTTPS request for the Google Search page and interprets the HTML/CSS/JavaScript sent back in response. DNS also operates here, converting `google.com` into an IP address before anything is transmitted.

Transport Layer – TCP takes the HTTP request, splits it into segments, numbers each one, and makes sure they all reach Google's server intact and in the right order. It's also responsible for the three-way handshake that opens the connection in the first place.

Internet Layer – IP packages each TCP segment into a packet, tagging it with the source IP (the user's device) and destination IP (Google's server). Routers rely on this layer to pass the packet across the many networks separating the user from Google's data center.

Network Access Layer – Turns IP packets into frames appropriate for whatever physical medium is being used — Wi-Fi radio waves or Ethernet electrical/optical signals — and manages the actual transmission over the local network hop.

Encapsulation as data moves through the layers

As the search request descends the stack on the sending side, each layer adds its own header on top of the data received from the layer above — a process known as encapsulation. The Application layer produces the raw HTTP request; the Transport layer wraps it in a TCP segment header (adding source/destination port numbers); the Internet layer wraps that in an IP packet header (adding source/destination IP addresses); and the Network Access layer wraps everything in a frame header (adding MAC addresses) before converting it into electrical, radio, or optical signals. On Google's server, the reverse happens — de-encapsulation — peeling away each header in turn until the original HTTP request is recovered.

Why TCP rather than UDP here

The Transport layer has a choice between TCP and UDP. Google Search specifically relies on TCP (carried inside HTTPS) instead of UDP, because a search-results page that arrives corrupted or incomplete simply isn't acceptable — the user needs the full, correctly ordered HTML. UDP offers no such delivery guarantees, so it's better suited to situations where speed outweighs perfect accuracy, like live video calls or the DNS lookups that happen before this very request.

3. The Role of DNS

Neither browsers nor routers understand domain names directly — traffic is routed using numeric IP addresses. The Domain Name System (DNS) acts as the Internet's directory service, translating names into addresses on our behalf.

- The browser first checks its own cache, and then the operating system's cache, for a recent record of google.com.
- If nothing is cached, a DNS resolver — typically operated by the ISP or a public service like 8.8.8.8 — is queried.
- The resolver moves through the DNS hierarchy: a root server directs it to the .com TLD server, which in turn directs it to Google's authoritative name server.
- The authoritative server responds with the actual IP address, for instance 142.250.193.174.
- That IP address is cached locally (until its TTL expires) and passed back to the browser so the connection can go ahead.

Without DNS, users would have to memorize a numeric address for every site they visit — which is exactly why DNS is often nicknamed the “phonebook of the Internet.”

Figure 3 (described): User's Device → Recursive Resolver (ISP) → Root Server → .com TLD Server → Google Authoritative Name Server, with the resolved IP passed back and cached at each step — showing hierarchical DNS resolution for google.com

DNS record types involved

Record Type	Purpose
A	Maps a domain name to an IPv4 address (e.g. google.com → 142.250.193.174)
AAAA	Maps a domain name to an IPv6 address
CNAME	Aliases one domain name to another, canonical domain name
NS	Specifies which name servers are authoritative for the domain
TTL	Not a record type itself, but a value telling resolvers how long to cache a record

4. HTTP: Requesting and Receiving the Page

Once the browser has resolved Google's IP address and opened a TCP connection, it uses HTTP (or its encrypted form, HTTPS) to actually retrieve the page:

- The browser sends a GET request — for example `GET /search?q=example HTTP/1.1` — along with headers such as `Host: www.google.com` and `User-Agent`.
- Google's web server processes that request and replies with an HTTP response: a status line (e.g. 200 OK), a set of response headers, and the body, which is the HTML document itself.
- The browser then parses the HTML and fires off further GET requests for any embedded resources — CSS stylesheets, JavaScript files, images, and fonts.
- Once enough of these resources have arrived, the browser renders the page on the screen.

A simplified request/response exchange

Roughly, this is what passes between browser and server:

Request (browser → server)

```
GET /search?q=example HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
```

Response (server → browser)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 54823
Set-Cookie: session_id=abc123; Secure; HttpOnly
<!DOCTYPE html>...(page content)...
```

HTTP versus HTTPS

In reality, a browser visiting Google never uses plain HTTP — it always uses HTTPS, which takes the same request/response cycle and wraps it inside a TLS (Transport Layer Security) encryption layer. Before any HTTP data is actually exchanged, the browser and server complete a TLS handshake to agree on encryption keys and confirm the server's identity via its certificate. This stops anyone eavesdropping on the network from reading or altering the search query or its results — which matters, since search terms can expose fairly sensitive personal information.

Common HTTP status codes

Code	Meaning	Example on google.com
200	OK – request succeeded, content returned	Search results page loads normally
301 / 302	Redirect to another URL	google.com redirecting to www.google.com
304	Not modified – use cached version	Re-loading a cached logo image
404	Requested resource not found	A broken or mistyped search link
500	Internal server error	A temporary fault on Google's servers

5. How TCP Delivers Data Reliably

HTTP depends entirely on TCP underneath it to guarantee that requests and responses arrive correctly. TCP achieves that reliability through a set of mechanisms:

Three-way handshake – SYN, SYN-ACK, and ACK segments set up a connection before any actual data changes hands, so both ends agree they're ready to communicate.

Sequence numbers – Every byte sent is numbered, which lets the receiver put segments back in the right order even if they don't arrive in that order.

Acknowledgements (ACKs) – The receiver confirms what it has received; a missing ACK is a signal that a segment went missing.

Retransmission – If an expected ACK doesn't show up within a timeout window, TCP resends the segment that was lost.

Flow control – A sliding-window mechanism stops a fast sender from overwhelming a slower receiver.

Checksums – Used to catch corrupted data so it can be thrown away and requested again.

6. Client–Server or Peer-to-Peer?

Google Search is built on a Client–Server architecture. The user's browser (the client) is always the one that initiates a request, while Google runs centralized, dedicated data-center servers that respond to those requests, maintain the search index, and return results. Clients never talk to one another directly to fetch results — every single request passes through Google's own infrastructure. That's the defining feature of a Client–Server system, in contrast to a Peer-to-Peer (P2P) system such as BitTorrent, where each participating node can act as both client and server, and content moves directly between peers with no central authority involved.

Aspect	Client–Server (Google Search)	Peer-to-Peer (e.g. BitTorrent)
Roles	Fixed – client requests, server responds	Interchangeable – every node is both client and server
Point of control	Centralized at Google's data centers	Decentralized across participating peers
Scalability approach	Achieved by adding more/larger servers	Achieved automatically as more peers join
Single point of failure	Yes, if the server/data center goes down	No, network survives individual peer loss
Typical use	Search engines, banking, email	File sharing, some blockchain networks

7. HTTP versus FTP

Aspect	HTTP	FTP
Purpose	Transfers web pages and resources for browsing/display	Transfers files between client and server for storage/download
Connections used	A single connection carries both requests and data	Two separate connections: control (port 21) and data (port 20)
State	Stateless by default (state added via cookies)	Stateful – the session persists for the duration of the connection
Authentication	Often not required for public content	Typically requires a username and password
Typical use case	Loading a webpage such as google.com	Uploading a website's files to a hosting server

It's worth noting that FTP's separate control and data connections can run in two modes: active mode, where the server opens a new connection back to the client to deliver data, and passive mode, where the client initiates both connections — the latter is friendlier to firewalls and has become the default in most modern FTP clients. HTTP has no equivalent distinction, since it relies on a single, simpler connection model — one reason it became the go-to protocol for everyday browsing, while FTP remains mainly useful for bulk file transfer and server administration.

8. How Cookies Improve the User Experience

HTTP is stateless by nature, meaning the server has no built-in memory of a user from one request to the next. Cookies solve this by letting the server store a small piece of data in the user's browser, which then gets sent back automatically with every later request to that same domain.

Real-world example: When a user signs into their Google account and performs a search, Google sets a session cookie. That cookie keeps the user logged in as they move from Search to Gmail to YouTube, without having to re-enter credentials on every page, and it also lets Google remember preferences such as language and safe-search settings across future visits.

9. End-to-End Communication Flow

Putting it all together, loading google.com follows this overall sequence between the browser, the DNS resolver, and Google's web server:

- 1. Browser asks the DNS resolver to resolve google.com
- 2. Resolver returns the IP address
- 3. Browser sends a TCP SYN to the server
- 4. Server responds with SYN-ACK
- 5. Browser sends ACK, and the connection is open
- 6. Browser sends the HTTP GET / request
- 7. Server replies with HTTP 200 OK and the HTML page
- 8. Browser parses the HTML, CSS, and JavaScript
- 9. The page is rendered on screen

Figure 2: Complete browser-to-server communication flow for loading google.com

10. Conclusion

Getting a single page like google.com onto the screen depends on a tightly coordinated stack of application-layer protocols operating quietly in the background. DNS converts a memorable name into a routable address, TCP makes sure the data survives the trip intact, IP charts the path across networks, and HTTP structures the actual exchange between browser and server. Cookies then add a layer of statefulness on top of a protocol that is inherently stateless, making personalization and persistent logins possible. Understanding these application-layer protocols matters because they form the direct interface between users and the Internet's infrastructure — every search, login, and page load depends on them working correctly, quickly, and securely. Without this layered design, the modern web, built on billions of HTTP requests every day, simply couldn't scale or stay reliable.

11. References

Kurose, J. F., & Ross, K. W. – Computer Networking: A Top-Down Approach

Forouzan, B. A. – Data Communications and Networking

Mozilla Developer Network (MDN Web Docs) – HTTP documentation

IETF RFC 793 – Transmission Control Protocol

IETF RFC 1035 – Domain Names: Implementation and Specification